1 - Fundamentos

Tabla de contenidos

1 Introducción 1
2 Funciones puras 1
  2.1 Efectos secundarios 2
3 Ciudadanos de primera clase 3
  3.1 Funciones de orden superior 5
  3.2 Atributos de una función 7
4 Funciones anónimas 9
    4.0.a Usos de funciones anónimas 9
5 Funciones variádicas 12
  5.1 Cantidad variable de argumentos posicionales `*args` 12
  5.2 Cantidad variable de argumentos nombrados `**kwargs` 15
  5.3 Combinando `*args` y `**kwargs` 17
6 *Closures* 18

1 Introducción

La **programación funcional** es un paradigma de programación que se centra en el uso de **funciones puras** y en concebir la computación como la evaluación de funciones. En lugar de dar instrucciones paso a paso que cambian variables o estados (como ocurre en la programación imperativa), la idea es construir programas a partir de funciones que transforman datos.

Existen lenguajes diseñados específicamente para la programación funcional (como Haskell), pero Python no es uno de ellos. Python es un lenguaje multiparadigma, lo que significa que nos permite combinar diferentes estilos de programación. Por este motivo, la programación funcional en Python no suele ser el enfoque principal, pero puede ser muy útil para escribir código más claro, conciso y fácil de probar.

2 Funciones puras

Una función es pura cuando su salida depende únicamente de los valores de entrada y no produce ningún efecto secundario o colateral (*side effect*, en inglés).

La función `sumar`, que calcula y devuelve la suma de dos números, es un ejemplo de función pura: su resultado depende solo de sus argumentos y no genera efectos colaterales.

```
def sumar(x, y):
    return x + y

sumar(3, 11)
```

```
14
```

En cambio, la función `agregar` no es una función pura. Esto se debe a que modifica un objeto global, lo que se conoce como un efecto secundario. Además, el valor de su salida no depende únicamente de la entrada, sino también de un estado global: la cantidad de elementos en `lista`.

```
lista = []

def agregar(x):
    """Agrega el elemento `x` al final de `lista` y devuelve la longitud de
`lista`"""
    lista.append(x)
    return len(lista)
```

```
agregar("azucar")
```

```
1
```

```
agregar("flores")
```

```
2
```

```
agregar("colores")
```

```
3
```

```
lista
```

```
['azucar', 'flores', 'colores']
```

2.1 Efectos secundarios

Un efecto secundario (*side effect*) es cualquier cambio de estado observable que ocurre fuera del ámbito local de una función. En otras palabras, se trata de una modificación del entorno externo de la función que va más allá de simplemente devolver un valor.

Algunos ejemplos de *side effects* son:

- Modificar una variable global o un objeto mutable.
- Imprimir en la consola.
- Escribir en un archivo.
- Realizar una llamada a una API o a una base de datos.

Las funciones con efectos secundarios pueden ser problemáticas porque, al modificar elementos externos, hacen que el código sea impredecible y difícil de probar.

En el ejemplo de la función `agregar` que creamos anteriormente, no es posible predecir el valor de salida para un valor de entrada determinado.

Por eso, la programación funcional promueve el uso de funciones puras, que no producen efectos secundarios. De esta manera, con las mismas entradas siempre se obtiene la misma salida, logrando un código más confiable, predecible y sencillo de mantener.

3 Ciudadanos de primera clase

Definamos otra función muy sencilla, `restar`, que calcula y devuelve la diferencia entre dos objetos.

```
def restar(x, y):
    return x - y

restar(10, 5)
```

```
5
```

Podemos observar que esta función es un **objeto** de tipo `function`.

```
print(type(restar))
print(restar)
restar
```

```
<class 'function'>
<function restar at 0x7f71cee62020>
<function __main__.restar(x, y)>
```

Al imprimir la función, Python nos muestra su nombre y la **dirección de memoria** donde está almacenada (en formato hexadecimal). En cambio, al mostrar su representación, obtenemos información adicional: el módulo en el que fue definida (en este caso `__main__`) y la lista de parámetros que recibe (`x` e `y`).

Dado que la función `restar` es un objeto de Python, podemos asignarla a una nueva variable y realizar una llamada utilizando esa nueva etiqueta en vez de la original.

```
resta_especial = restar
resta_especial(10, 5)
```

```
5
```

Notemos que `resta_especial` **no es una nueva función**; es solamente una nueva referencia a la función antes definida.

```
resta_especial # muestra 'restar', no 'resta_especial'
```

```
<function __main__.restar(x, y)>
```

```
resta_especial is restar
```

```
True
```

En Python, las funciones son ciudadanos de primera clase. Esto significa que son objetos, al igual que las cadenas o los números. Por lo tanto, todo lo que se puede hacer con una cadena o un número también puede hacerse con una función.

Por ejemplo, se pueden almacenar dentro de una lista junto con otros objetos de distintos tipos:

```
popurri = [128, restar, None]
print(popurri[0])
print(popurri[1])
print(popurri[2])
```

```
128
<function restar at 0x7f71cee62020>
None
```

Incluso una función puede ser almacenada como valor en un diccionario:

```
mapeo = {
    "sum": sumar,
    "sub": restar,
}
```

Luego, se las puede usar de la siguiente manera:

```
mapeo["sum"](25, 4)
```

```
29
```

```
mapeo["sub"](25, 4)
```

```
21
```

3.1 Funciones de orden superior

Como cualquier objeto de Python, una función puede ser pasada como argumento de otra función. Debajo definimos dos funciones muy simples. Una imprime un mensaje de bienvenida y la otra uno de despedida.

```
def bienvenida():
    print("¡Hola!")

def despedida():
    print("¡Chau!")


bienvenida()
despedida()
```

```
¡Hola!
¡Chau!
```

Se puede definir otra función, que llamaremos `externa` (del inglés *outer function*), que tiene un único parámetro `interna`. En su cuerpo, la función `externa` llama a la función `interna` y devuelve lo que sea que `interna` devuelva.

```
def externa(interna):
    return interna()
```

De este modo, si llamamos a `externa` pasándole como argumento a `bienvenida`, se imprimirá `¡Hola!`; y si lo hacemos con `despedida`, se imprimirá `¡Chau!`.

```
externa(bienvenida)
```

```
¡Hola!
```

```
externa(despedida)
```

```
¡Chau!
```

Como ni `bienvenida` ni `despedida` devuelven nada, lo mismo ocurre con `externa` en los dos ejemplos anteriores.

A esta función podemos pasarle cualquier función que pueda ser llamada sin ningún argumento. Por ejemplo:

```
def crear_lista():
    return []

externa(crear_lista)
```

```
[]
```

También es posible que una función devuelva como resultado otra función. La función `fabricar` construye y devuelve una función que computa la suma de dos objetos.

```
def fabricar():
    def interna(x, y):
        return x + y
    return interna

# La llamada a 'fabricar' genera y devuelve una función
f = fabricar()

# La función obtenida puede ser tratada como cualquier otra función
f(10, 15)
```

```
25
```

Una función que fabrica otras funciones puede recibir parámetros que luego son utilizados dentro de la función interna. En el bloque siguiente, la función `crear_multiplicador` recibe un parámetro `x`, que define el valor por el cual se multiplicará el argumento de la función interna que se devuelve.

```
def crear_multiplicador(x):
    def interna(y):
        return y * x
    return interna
```

Así, es posible crear funciones para duplicar, triplicar, etc.

```
duplicar = crear_multiplicador(2)
triplicar = crear_multiplicador(3)

print(duplicar(5))
print(triplicar(5))
print(triplicar(18))
```

```
10
15
54
```

> ℹ Observación 👀
>
> Cada vez que se invoca la función `fabricar`, se crea y devuelve una **nueva** función. Por eso, el resultado de la comparación en el siguiente bloque es `False`.

```
f1 = fabricar()
f2 = fabricar()
print(f1 is f2)
```

```
False
```

> ℹ *Function factory* 🏭
>
> A las funciones que crean y devuelven funciones se las conoce como **fábrica de funciones**, del inglés *function factory*.

3.2 Atributos de una función

En Python, las funciones también cuentan con atributos, del mismo modo que otros objetos. En el siguiente ejemplo definimos la función `resolvente`, que recibe las constantes `a`, `b` y `c` de un polinomio de segundo grado, calcula sus raíces usando la fórmula resolvente y las devuelve en una tupla.

```
def resolvente(a, b, c):
    discriminante = b ** 2 - 4 * a * c
    x0 = (-b + (discriminante) ** 0.5) / (2 * a)
    x1 = (-b - (discriminante) ** 0.5) / (2 * a)

    return x0, x1

resolvente(2, 5, -3)
```

```
(0.5, -3.0)
```

A través del atributo especial `__code__` es posible consultar ciertos atributos o detalles internos de una función:

```
print(resolvente.__code__.co_argcount) # Cantidad de argumentos
print(resolvente.__code__.co_name)     # Nombre de la función
print(resolvente.__code__.co_varnames) # Variables en el ámbito local
```

```
3
resolvente
('a', 'b', 'c', 'discriminante', 'x0', 'x1')
```

Acceder a la información de una función a través de `__code__` puede resultar poco práctico, ya que los atributos disponibles son técnicos y no siempre coinciden directamente con lo que solemos necesitar (por ejemplo, obtener solo los nombres de los argumentos).

Para facilitar esta tarea, la librería estándar de Python incluye el módulo `inspect`, que ofrece herramientas más claras e intuitivas para explorar los atributos y detalles de una función.

A modo ilustrativo tomemos la función `signature`, que devuelve un objeto que representa la **firma** de la función `resolvente`.

```
import inspect

firma = inspect.signature(resolvente)
firma
```

```
<Signature (a, b, c)>
```

A partir de esta firma podemos consultar distintos aspectos de los parámetros, como sus valores por defecto:

```
firma.parameters["a"].default # 'a' no tiene asignado un valor por defecto
```

```
inspect._empty
```

Finalmente, `inspect` también permite acceder al código fuente de la función en forma de cadena de texto:

```
print(inspect.getsource(resolvente))
```

```
def resolvente(a, b, c):
    discriminante = b ** 2 - 4 * a * c
    x0 = (-b + (discriminante) ** 0.5) / (2 * a)
    x1 = (-b - (discriminante) ** 0.5) / (2 * a)

    return x0, x1
```

4 Funciones anónimas

La programación funcional se basa en llamar funciones y pasarlas, por lo que, naturalmente, puede implicar definir muchas funciones. En Python, además de usar `def`, podemos crear funciones **anónimas** de forma rápida con una **expresión *lambda***.

La sintaxis es la siguiente:

```
lambda <argumentos>: <expresión>
```

y devuelve como resultado una función anónima. Un ejemplo es el siguiente:

```
lambda x, y: x + y
```

```
<function __main__.<lambda>(x, y)>
```

Como la función anónima que acabamos de crear no fue asignada a ninguna variable, ya no podemos usarla.

Una posibilidad es invocarla inmediatamente al momento de su creación:

```
(lambda x, y: x + y)(7, 15)
```

```
22
```

Otra opción es asignarla a una variable para poder llamarla más adelante:

```
sumar = lambda x, y: x + y
sumar(7, 15)
```

```
22
```

> ℹ Ausencia de `return`
>
> A diferencia de las funciones definidas con `def`, las expresiones *lambda* no requieren la sentencia `return`. De forma implícita, siempre devuelven el resultado de la única expresión que contienen.

4.0.a Usos de funciones anónimas

En ninguno de los ejemplos anteriores parece que obtengamos alguna ventaja frente a usar `def` para definir una función. De hecho, da la impresión de que estamos complicando el código innecesariamente.

Lo cierto es que las funciones anónimas no están pensadas para emplearse de la manera expuesta en nuestros ejemplos. Su uso principal es en **operaciones simples y puntuales**, cuando no resulta práctico definir una función regular con `def`.

Un caso de uso típico de la funciones anónimas es cuando se tiene que pasar una función como argumento de otra función.

Supongamos que queremos ordenar la siguiente lista de refranes según diferentes criterios.

```
refranes = [
    "Al mal tiempo, buena cara",
    "Perro que ladra no muerde",
    "A caballo regalado no se le miran los dientes",
    "Cada loco con su tema",
    "El que mucho abarca, poco aprieta",
    "Más vale pájaro en mano que cien volando",
]
```

Por defecto, la función `sorted` ordena una lista de cadenas de manera alfabética.

```
sorted(refranes)
```

```
['A caballo regalado no se le miran los dientes',
 'Al mal tiempo, buena cara',
 'Cada loco con su tema',
 'El que mucho abarca, poco aprieta',
 'Más vale pájaro en mano que cien volando',
 'Perro que ladra no muerde']
```

Si quisiéramos ordenar los refranes por su longitud, podemos usar el argumento opcional `key` de `sorted`. Este argumento recibe una función que, **aplicada a cada elemento**, devuelve el valor a utilizar en el ordenamiento. En nuestro caso, basta con usar `len`, ya que solo nos interesa la cantidad de caracteres de cada cadena.

```
sorted(refranes, key=len)
```

```
['Cada loco con su tema',
 'Al mal tiempo, buena cara',
 'Perro que ladra no muerde',
 'El que mucho abarca, poco aprieta',
 'Más vale pájaro en mano que cien volando',
 'A caballo regalado no se le miran los dientes']
```

Así, obtenemos una lista donde las frases se ordenan según la cantidad de caracteres.

¿Y si quisiéramos ordenarlos según la **cantidad de palabras**? Para eso necesitamos una función que reciba una cadena, la divida en palabras y cuente cuántas tiene.

Sin funciones anónimas podríamos hacer lo siguiente:

```
def contar_palabras(x):
    return len(x.split())

sorted(refranes, key=contar_palabras)
```

```
['Al mal tiempo, buena cara',
 'Perro que ladra no muerde',
 'Cada loco con su tema',
 'El que mucho abarca, poco aprieta',
 'Más vale pájaro en mano que cien volando',
 'A caballo regalado no se le miran los dientes']
```

En cambio, con una función anónima podemos escribir todo el programa en una sola línea:

```
sorted(refranes, key=lambda x: len(x.split()))
```

```
['Al mal tiempo, buena cara',
 'Perro que ladra no muerde',
 'Cada loco con su tema',
 'El que mucho abarca, poco aprieta',
 'Más vale pájaro en mano que cien volando',
 'A caballo regalado no se le miran los dientes']
```

De esta forma el código es más **conciso** y evitamos definir funciones "descartables" que no volverán a usarse.

> ℹ Origen del nombre *lambda* λ✨
>
> El término *lambda* proviene del cálculo lambda, un sistema formal de lógica matemática para expresar cálculos basados en la abstracción y aplicación de funciones.
>
> Se le dio ese nombre porque Alonzo Church, creador del cálculo *lambda* en la década de 1930, usó la letra griega λ para denotar la operación de abstracción de funciones.

Funciones anónimas sin parámetros

Una función *lambda* normalmente recibe uno o múltiples parámetros, pero no es obligatorio, por lo que es posibile escribir una función anónima sin parámetros:

```
crear_numero_magico = lambda: 128
crear_numero magico()
```

```
128
```

La función anónima `crear_numero_magico` es equivalente a la siguiente función

```
def f():
    return 128
```

5 Funciones variádicas

Las funciones variádicas son funciones que pueden recibir una cantidad variable de argumentos.

A lo largo de estos apuntes hemos utilizado funciones variádicas en tantísimas oportunidades. Un ejemplo de función variádica es `print`, que acepta tantos argumentos posicionales como necesitemos.

```
print("Primero")
```

```
Primero
```

```
print("Primero", "segundo")
```

```
Primero segundo
```

```
print("Primero", "segundo", "tercero")
```

```
Primero segundo tercero
```

Afortunadamente, no solo las funciones *built-in* pueden ser variádicas, sino que también podemos implementarlas nosotros mismos.

5.1 Cantidad variable de argumentos posicionales `*args`

Supongamos que queremos una función que recibe una cantidad arbitraria de gustos de helado e imprime un mensaje como si lo agregase a un pedido. Por ejemplo:

```
armar_pedido("Dulce de leche")
```

```
Agregando 'Dulce de leche'
```

```
armar_pedido("Dulce de leche", "Sambayón", "Frutos del bosque")
```

```
Agregando 'Dulce de leche'
Agregando 'Sambayón'
Agregando 'Frutos del bosque'
```

Una posible implementación para tal función es:

```
def armar_pedido(gusto_1=None, gusto_2=None, gusto_3=None):
    if gusto_1 is not None:
        print(f"Agregando '{gusto_1}'")
    if gusto_2 is not None:
        print(f"Agregando '{gusto_2}'")
    if gusto_3 is not None:
        print(f"Agregando '{gusto_3}'")
```

Aunque funciona en los ejemplos anteriores, esta solución está lejos de ser ideal. Requiere definir un argumento separado para cada gusto, asignarle un valor por defecto y luego verificar si es distinto de `None` antes de agregarlo al pedido.

Además, el código resulta repetitivo y restrictivo: solo permite un máximo de tres gustos.

En cambio, podemos crear una función que reciba una cantidad arbitraria de argumentos posicionales. Para ello se utiliza un **argumento especial** precedido por un asterisco (*), lo que le permite recibir una cantidad arbitraria de valores no nombrados.

```
def armar_pedido(*args):
    for gusto in args:
        print(f"Agregando '{gusto}'")
```

```
armar_pedido("Dulce de leche", "Sambayón", "Frutos del bosque", "Menta
granizada")
```

```
Agregando 'Dulce de leche'
Agregando 'Sambayón'
Agregando 'Frutos del bosque'
Agregando 'Menta granizada'
```

```
armar_pedido("gusto 1", "gusto 2", "gusto 3", "gusto 4", "gusto 5", "gusto 6",
"gusto 7")
```

```
Agregando 'gusto 1'
Agregando 'gusto 2'
Agregando 'gusto 3'
Agregando 'gusto 4'
Agregando 'gusto 5'
Agregando 'gusto 6'
Agregando 'gusto 7'
```

Por convención, este argumento suele escribirse como `*args`, aunque en realidad el nombre del argumento puede ser cualquiera que resulte apropiado. En nuestro caso, resulta más intuitivo usar `*gustos`, y la función quedaría así:

```
def armar_pedido(*gustos):
    for gusto in gustos:
        print(f"Agregando '{gusto}'")
```

```
armar_pedido("gusto 1", "gusto 2", "gusto 3", "gusto 4", "gusto 5", "gusto 6",
"gusto 7")
```

```
Agregando 'gusto 1'
Agregando 'gusto 2'
Agregando 'gusto 3'
Agregando 'gusto 4'
Agregando 'gusto 5'
Agregando 'gusto 6'
Agregando 'gusto 7'
```

ℹ Qué hay debajo de *args 🔍

Python agrupa automáticamente en una tupla los valores pasados mediante el argumento especial *args. Esto permite acceder a todos los argumentos como miembros de una colección inmutable.

```
def fun(*args):
    print(len(args))
    print(args)
    print(type(args))

fun("que", "es", "esto", True, None)
```

```
5
('que', 'es', 'esto', True, None)
<class 'tuple'>
```

5.2 Cantidad variable de argumentos nombrados **kwargs

Así como recibimos una cantidad arbitraria de argumentos posicionales, también podemos recibir una cantidad arbitraria de argumentos nombrados.

En este caso, se utilizan dos asteríscos (**) en vez de uno (*) en la definición de los parámetros de la función.

La convención es usar el nombre **kwargs, pero también es válido usar cualquier otro nombre que sea adecuado en nuestro contexto.

Comencemos por un ejemplo elemental, que solo imprime el objeto kwargs y su tipo:

```
def ejemplo(**kwargs):
    print(kwargs)
    print(type(kwargs))

ejemplo(nombre="Mariano", apellido="Pérez")
```

```
{'nombre': 'Mariano', 'apellido': 'Pérez'}
<class 'dict'>
```

Cuando usamos una cantidad variable de argumentos nombrados, Python los agrupa en un diccionario, ya que esta estructura permite asociar cada nombre con su valor de forma natural.

Dentro de la función, se puede manipular al diccionario kwargs como a cualquier otro diccionario de Python.

Imaginemos, por ejemplo, una función que registra información de distintos departamentos. En este caso, no sabemos de antemano qué atributos se van a proporcionar, pero sí sabemos que ciertos atributos deben contar con un valor por defecto si no se especifican.

```
def registrar_propiedad(**kwargs):
    print("Diccionario original:")
    print(kwargs)

    # Si no se especifica la cantidad de cocheras, se pone 0 por defecto
    if "cochera" not in kwargs:
        kwargs["cochera"] = 0

    # Si no se especifica la ciudad, se pone 'Desconocido' por defecto
    if "ciudad" not in kwargs:
        kwargs["ciudad"] = "Desconocido"

    return kwargs
```

Cuando no se especifican la cantidad de cocheras, la función nos devuelve un diccionario donde la cantidad de cocheras es 0.

```
datos = registrar_propiedad(ambientes=2, ciudad="Rosario")

print("\nDiccionario sanitizado")
print(datos)
```

```
Diccionario original:
{'ambientes': 2, 'ciudad': 'Rosario'}

Diccionario sanitizado
{'ambientes': 2, 'ciudad': 'Rosario', 'cochera': 0}
```

Si los atributos requeridos son especificados, se devuelve el diccionario sin cambios.

```
datos = registrar_propiedad(ambientes=2, ciudad="Santa Fe", cochera=2)

print("\nDiccionario sanitizado")
print(datos)
```

```
Diccionario original:
{'ambientes': 2, 'ciudad': 'Santa Fe', 'cochera': 2}

Diccionario sanitizado
{'ambientes': 2, 'ciudad': 'Santa Fe', 'cochera': 2}
```

Y si se pasan otros atributos, también se incluyen en la salida.

```
datos = registrar_propiedad(ambientes=4, dormitorios=2)

print("\nDiccionario sanitizado")
print(datos)
```

```
Diccionario original:
{'ambientes': 4, 'dormitorios': 2}

Diccionario sanitizado
{'ambientes': 4, 'dormitorios': 2, 'cochera': 0, 'ciudad': 'Desconocido'}
```

5.3 Combinando *args y **kwargs

Las funciones en Python pueden recibir simultáneamente una cantidad variable de argumentos posicionales y nombrados. Para lograrlo, se combinan `*args` y `**kwargs`. Es importante recordar que, al definir la función, `*args` debe colocarse antes que `**kwargs`, ya que los argumentos posicionales siempre se pasan antes que los nombrados.

```
def superfuncion(*args, **kwargs):
    for arg in args:
        print(f"Me pasaron el argumento posicional '{arg}'")

    for key, value in kwargs.items():
        print(f"Me pasaron el argumento con nombre '{key}' y valor '{value}'")

superfuncion(True, 64, nombre="Elsa", apellido="Pato")
```

```
Me pasaron el argumento posicional 'True'
Me pasaron el argumento posicional '64'
Me pasaron el argumento con nombre 'nombre' y valor 'Elsa'
Me pasaron el argumento con nombre 'apellido' y valor 'Pato'
```

Si se intenta pasar un argumento posicional (sin nombre) después de un argumento nombrado, obtendríamos un error:

```
superfuncion(True, nombre="Elsa", apellido="Pato", 64)
```

```
    superfuncion(True, nombre="Elsa", apellido="Pato", 64)
                                                       ^
SyntaxError: positional argument follows keyword argument
```

> **i** ¿Y para qué me sirven? 🤔
>
> A primera vista, los ejemplos de `*args` y `**kwargs` pueden dar la impresión de que estas herramientas solo complican la escritura del código. Sin embargo, su verdadero valor aparece al trabajar en programas más complejos, donde se vuelven fundamentales para simplificar la lógica y aportar flexibilidad en la resolución de una gran variedad de problemas.
>
> Ya llegaremos...

> **i** Es solo una convención 🤝
>
> Para reforzar que los nombres `*args` y `**kwargs` son solamente una convención, podríamos escribir la función `superfuncion` como:
>
> ```
> def superfuncion(*posicionales, **nombrados):
>     for arg in posicionales:
>         print(f"Me pasaron el argumento posicional '{arg}'")
>
>     for key, value in nombrados.items():
>         print(f"Me pasaron el argumento con nombre '{key}' y valor
> '{value}'")
> ```
>
> y funcionaría de igual modo.

6 *Closures*

En la Sección 3.1 vimos el siguiente ejemplo:

```
def crear_multiplicador(x):
    def interna(y):
        return y * x
    return interna

duplicar = crear_multiplicador(2)
triplicar = crear_multiplicador(3)

print(duplicar(5))
print(triplicar(5))
```

```
10
15
```

La función `crear_multiplicador` es una fábrica de funciones que devuelve otra función que se encarga de realizar la multiplicación. Lo interesante de esta implementación es que la función

interna solo recibe uno de los dos valores necesarios para la multiplicación; el otro queda que fijado cuando se ejecuta la función externa `crear_multiplicador`.

Para que `duplicar` y `triplicar` funcionen correctamente, ambas funciones internas deben conservar acceso al entorno en el que está definido el valor de `x`. Un ***closure*** combina una función con el entorno de variables en el que fue definida. Esto permite que la función conserve acceso a esas variables incluso después de que termine la ejecución de la función externa.

```
def crear_multiplicador(x):
    def interna(y):
        return y * x
    return interna

duplicar = crear_multiplicador(2)
triplicar = crear_multiplicador(3)

print(duplicar(5))
print(triplicar(5))
```

```
10
15
```

El siguiente ejemplo hace aún más evidente el funcionamiento de este mecanismo.

Dentro del cuerpo de la *function factory* `externa`, se define `valor` con el número `256`. Luego, la función interna hace uso de esta variable `valor` dentro de `print`.

```
def externa():
    valor = 256
    def closure():
        print(f"¡El valor es: {valor}!")
    return closure

revelar_numero = externa()
revelar_numero()
```

```
¡El valor es: 256!
```

Aunque desde fuera no podemos acceder directamente a `valor`:

```
valor
```

```
NameError: name 'valor' is not defined
```

la función interna sí puede hacerlo tantas veces como sea necesario:

```
revelar_numero()
revelar_numero()
revelar_numero()
```

```
¡El valor es: 256!
¡El valor es: 256!
¡El valor es: 256!
```

Para finalizar, veamos un ejemplo similar al anterior, pero donde el valor de la variable `numero` es desconocido para nosotros. Dicho valor se genera de manera aleatoria cuando se ejecuta la fábrica de funciones `crear_funcion`.

```
import random

def crear_funcion():
    numero = random.randint(1, 1000)
    def closure():
        print("El valor es...", numero)
    return closure

reveladora = crear_funcion()
```

Luego, sin importar cuántas veces llamemos a `reveladora`, el mensaje será siempre el mismo, ya que el valor de `numero` se definió una sola vez en el momento en que se creó la función.

```
reveladora()
```

```
El valor es... 393
```

```
reveladora()
```

```
El valor es... 393
```

Una dosis de precisión 🎯 😱

A menudo se dice que un ***closure*** es una función. Así, en el siguiente ejemplo, `duplicar` suele describirse como un *closure*:

```
def crear_multiplicador(x):
    def interna(y):
        return y * x
    return interna

duplicar = crear_multiplicador(2)
```

Sin embargo, esta forma de expresarlo es algo imprecisa. Más precisamente, un *closure* combina una función con el entorno de variables en el que fue definida. Esto permite que la función conserve acceso a esas variables incluso después de que termine la ejecución de la función externa. En el ejemplo, `interna` conserva acceso a `x` aun después de que finaliza la ejecución de `crear_multiplicador`.

`object of type 'closure' is not subsettable` 😵

Si en R intentamos seleccionar filas o columnas de `data` sin haberle asignado un objeto previamente, obtendremos el siguiente error:

```
Error in data[1] : object of type 'closure' is not subsettable
```

Esto ocurre porque `data` es en realidad una función en R. En este lenguaje, el tipo de los objetos función se denomina `closure`, haciendo referencia la capacidad que tienen las funciones de acceder a valores del ambiente donde fueron definidas.